Efficient Concept Drift Detection: A Meta Feature Selection Approach

Zelong Liu
Xi'an Jiaotong-Liverpool University
China
zelong.liu23@student.xjtlu.edu.cn

Pingfan Wang
Northumbria University
United Kingdom
pingfan2.wang@northumbria.ac.uk

Nanlin Jin
Xi'an Jiaotong-Liverpool University
China
nanlin.jin@xjtlu.edu.cn

Abstract

Concept drift in data streams significantly impacts predictive modelling, as the underlying distribution of the training sample evolves, often leading to increased error rates and degraded model performance. When dealing with big data which has many features and data speed is high, traditional concept drift detection methods might delay in analyzing and detecting in time. This paper introduces an improved drift detection model utilizing multivariate analysis methods for feature selection, thereby enhancing the model's ability to detect drift more efficiently. Our approach analysis various parameters of the data stream to select the most important features and conduct conduct drift detection based on the selected features. Experimental results demonstrate that this feature-selected drift detection model not only maintains classification performance, but also significantly reducing computational overhead. This predictive framework is particularly valuable in scenarios where large data streams require real-time analysis and where computational resources are limited, providing a practical solution for maintaining robust model performance in dynamic environments.

CCS Concepts

• **Computing methodologies** → Artificial intelligence; Natural language processing; Information extraction; • **Computing methodologies** → Machine learning; Machine learning algorithms; Feature selection.

Keywords

Data Stream Mining, Concept Drift Detection, Meta feature, Feature Selection

ACM Reference Format:
Zelong Liu, Pingfan Wang, and Nanlin Jin. 2024. Efficient Concept Drift Detection: A Meta Feature Selection Approach. In *2024 the 7th International Conference on Information Science and Systems (ICISS) (ICISS 2024), August 14–16, 2024, Edinburgh, United Kingdom.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3700706.3700721

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICISS 2024, August 14–16, 2024, Edinburgh, United Kingdom
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1756-7/24/08
https://doi.org/10.1145/3700706.3700721

1 Introduction

In the dynamic landscape of data-driven technologies, the precision of predictive models is constantly challenged by concept drift—unpredictable shifts in the underlying data distribution that can significantly degrade model performance. These shifts are especially common in real-time data streams such as sensor readings, online user activities, and financial signals, which are inherently prone to distribution fluctuations [1]. Such environments demand the deployment of advanced adaptive learning strategies to effectively detect and respond to these changes.

In the context of big data environments characterized by high dimensionality and rapid data velocity, traditional concept drift detection methods often struggle to keep pace, resulting in delayed analysis and detection [2]. These traditional methods are typically not designed to efficiently handle the sheer volume and complexity of modern data streams, where features are abundant and data arrives continuously at high speed [3].

To address these challenges, this paper proposes an innovative drift detection model that leverages multivariate analysis techniques for effective feature selection. By focusing on meta features that are most indicative of drift, our model significantly enhances the efficiency and responsiveness of drift detection. This improved model not only identifies the onset of concept drift more rapidly but also adapts more seamlessly to the evolving data landscape, ensuring that predictive models remain accurate and reliable even in the face of rapid data changes. This approach represents a critical advancement in the field of data stream analysis, offering a more robust solution for maintaining model performance in dynamic environments.

Our contributions are designed to provide a robust framework for predictive modeling, dynamically adjusting to changes within data streams to ensure sustained performance and ongoing relevance. These contributions include:

1. Enhanced Feature Selection and Dimension Reduction: We implement sophisticated feature selection and dimension reduction techniques, concentrating on critical meta-features. This streamlines model operations and bolsters responsiveness to novel data conditions.

2. Optimized Computational Resources for Concept Drift Detection: Our proposed framework drastically reduces the use of computational resources while maintaining high drift prediction accuracy. This is particularly advantageous in scenarios where such resources are scarce.

In summary, our paper underscores the necessity of advanced strategies in predictive modelling to contend with the complexities

introduced by concept drift in data streams. Through our comprehensive framework, we aim to set new standards for reliability and efficiency in adaptive learning systems.

2 RELATED WORK

2.1 Concept drift detection

In the dynamic field of predictive modeling, concept drift detection remains a pivotal challenge, necessitating robust mechanisms for maintaining model accuracy over time. This section of the literature review focuses on performance-based and distribution-based methods for detecting concept drift, summarizing key approaches and their implications [4].

Performance-Based Detection: primarily monitor changes in a model's accuracy, triggering drift detection when significant performance deviations occur. The Drift Detection Method (DDM) [5], developed by Gama et al., is a cornerstone in this area, utilizing overall prediction error rates to identify drifts. This method is complemented by the Early Drift Detection Method (EDDM) [6] which focuses on the distance between consecutive prediction errors, though it requires a minimum of 30 errors to function effectively, potentially delaying response to sudden drifts. Other notable methods include the Hoeffding's inequality-based Drift Detection Method (HDDM) [7] and the Fast Hoeffding Drift Detection Method (FHDDM) [8], which leverage statistical inequalities to define critical regions for drift detection, enhancing the responsiveness of the detection process.

Distribution-Based Detection: assess drift by measuring the dissimilarity between data distributions over time, offering advantages in scenarios involving unlabeled data. Techniques like the Kolmogorov-Smirnov (KS) test are widely used for this purpose, providing a statistical basis for comparing distributions from different data samples [9]. Moreover, advanced clustering techniques and modified versions of the KS test have been adapted to enhance the efficiency and effectiveness of drift detection in large data streams. These methods, while powerful, tend to demand higher computational resources compared to performance-based approaches.

Each approach offers unique benefits and challenges, underscoring the importance of selecting a suitable method based on specific requirements and data characteristics. For further reading and a deeper understanding of these methodologies, the original studies and reviews provide comprehensive insights and comparative analyses.

2.2 Feature Selection

Feature selection is a critical process in the construction and optimization of machine learning models, particularly important in the fields of data mining, statistics, and artificial intelligence. Its primary goal is to identify and select a subset of relevant features for use in model construction. This process not only simplifies the model to make it faster and less costly to operate but also improves the model's performance by eliminating irrelevant or partially relevant features that can cause the model to perform poorly on new data.

Feature selection techniques are broadly categorized into three groups: filter methods, wrapper methods, and embedded methods [10]. Each category has its methodologies and applications, influenced by their strengths and limitations.

Filter Methods: These methods are typically independent of machine learning algorithms. They evaluate features based on general characteristics like correlation with the target variable, using statistical tests for this purpose. The advantage of filter methods lies in their simplicity and speed, which makes them particularly useful for high-dimensional datasets.

Wrapper Methods: Unlike filter methods, wrapper methods evaluate subsets of variables to determine their collective impact on the performance of a predictive model. This approach involves selecting a set of features, constructing a model, and evaluating its performance iteratively. While more computationally expensive, wrapper methods often provide improved model performance because they consider the interaction between variables.

Embedded Methods: These methods perform feature selection as part of the model construction process and are specific to given learning algorithms. Techniques such as lasso and ridge regression are common examples, where feature selection is built into the model training process, making them efficient for both feature selection and model training.

The iterative feature selection process, as seen in machine learning applications, leverages both the filter and wrapper methods. This approach iteratively refines feature selection by evaluating the impact of adding or removing features based on model performance criteria. Such iterative methods help in dynamically adjusting to the needs of the model and the inherent characteristics of the data.

3 FS-DDM algorithm

We present an improved drift detection algorithm called **F**eature **S**election and **D**rift **D**etection **M**ethod (FS-DDM) which consists of the feature selection and drift detection method. The general algorithm structure is presented as Figure 1.

The sliding window will select part of the data from the general dataset and send to the feature selection model. The selected feature will be sent to the drift detection model, to conduct classification in our case. Then the classification performance will be evaluated and decide whether the feature selected this round can be pass on to the next round of classification.

We implement iterative feature selection using Linear Regression and XGBoost to address the challenges posed by high-dimensional data in drifting stream environments. This approach allows us to conduct dimension reduction effectively even with small-scale data streams.

For drift detection, we apply Adaptive Random Forest Regression and Adaptive XGBoost. These methods are designed to adaptively update their models in response to changes in the data stream, providing robustness against varying data conditions.

4 Experiment and Evaluation

4.1 Experiment Overview

We present a model combining feature selection and stream data mining algorithm. The model consists of four parts: Input data

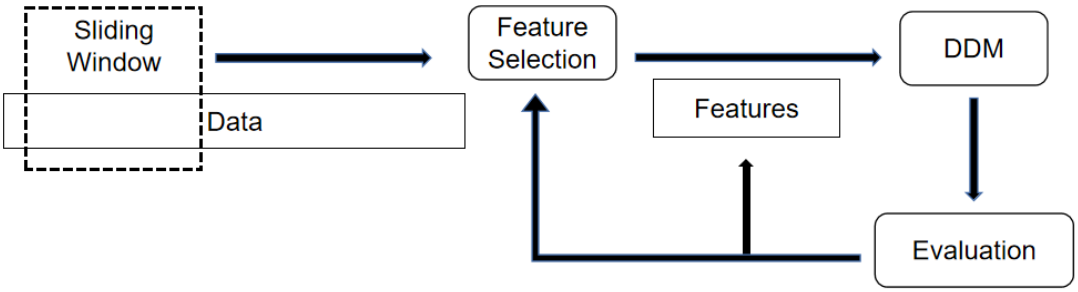


Figure 1: The workflow of proposed FS-DDM method

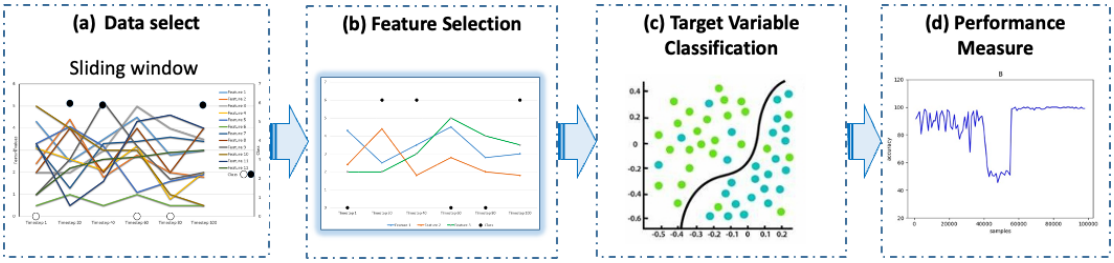


Figure 2: Experiment Design to test FS-DDM method

Table 1: Details of Datasets

Dataset	Dimension Size	Data Size	Drift Point	Abrupt Drift
ARGAWAL	9	[10000,100000,1000000]	40%	Yes
HYPER	10 & 20	[10000,100000,1000000]	40%	No

selection, Feature Selection, Target variable classification and Performance measure. Details shown in Figure 2.

4.2 Date Overview

In our study, we explore datasets ranging from 10,000 to 1,000,000 samples, with feature counts varying from 10 to 20, to assess the computational impact associated with varying dataset configurations. To effectively simulate a data stream on historical data, we utilize a sliding window approach. The data is generated using the scikit-multiflow framework [11], a specialized machine learning package for streaming data applications. We focus on two types of synthetic data streams:

- **AGRAWAL** [12]: This dataset is derived from the Agrawal generator, which simulates a data stream incorporating five nominal and three numerical features. It categorizes instances into two classes based on distinct functions.
- **HYPER** [13]: This dataset is generated using a random hyperplane generator and features fast incremental drifts. It involves a hyperplane in a d-dimensional space that frequently changes its position and orientation, challenging the adaptability of our models.

The dataset interpretation is listed in Table 1.

4.3 Feature Selection

Feature selection algorithms, including Linear Regression (LR) and XGBoost (XGB), are employed to reduce dimensionality. The XGB algorithm processes data batches provided by the sliding window, dividing each batch into two segments: 70% for training and 30% for testing. The training segment trains the LR algorithm to calculate feature importance. Concurrently, the LR algorithm utilizes the same training data for both training and feature selection processes. Subsequently, the importance of each feature is evaluated using the testing data to assess the efficacy of the selection. Details of the feature selection algorithm are outlined in Algorithm 1.

4.4 Target Variable Classification

Classification algorithms employed in this study include Adaptive Random Forest Regression (ARF) and Adaptive XGBoost (AXGB). Prior to training, the selected features are inputted into these two classification algorithms. This data is provided by the sliding window, which also supplies the training dataset previously used for feature selection. Following the training phase, the testing data is then used to evaluate the classification performance of both ARF and AXGB algorithms.

Algorithm 1 Feature selection algorithm

Input: Training data, Testing data, feature selection model
Train the feature selection model with all the features in Training data
Get importance of each features from the model and sort in **descending order**
Foreach: feature $\boldsymbol{f}$ and its importance $\boldsymbol{I}$ in Training data:
  Select all the features whose importance is higher than $\boldsymbol{I}$
  Train the model with these features
  Fit the model with Testing data and get classification accuracy
  If: accuracy > accuracy threshold
    Lift the importance threshold to $\boldsymbol{I}$
    Record $\boldsymbol{f}$ as one of the selected features
Output: the selected features, feature selection model

4.5 Performance measure

The performance of the combined algorithms—LR+ARF and XGB+AXGB—is assessed by comparing the classification outcomes with the true labels of each sample to determine classification accuracy. Additionally, the efficacy of feature selection is evaluated by calculating the valid selection rate based on the classification accuracy achieved in each round. If the classification accuracy of a particular batch falls below a predefined threshold, the feature selection for that round is deemed unsuccessful and marked as invalid. The overall valid selection rate is computed at the conclusion of the process. Detailed procedures and criteria for evaluation are outlined in Algorithm 2.

Algorithm 2 Feature Selection & Drift Detection algorithm

Input: Dataset
For: sliding window inside the dataset
 Select training Data, testing Data obtained by sliding window
  Select features with training data
 Classify the testing data with selected features
 Get classification accuracy of this round A
  If: A < threshold :
   Reset the classification model and the feature selection model
   Record this round of feature selection as invalid
   Record the accuracy of classification
Output: Performance evaluation

The threshold is a user-defined parameter that can be adjusted to suit different scenarios according to user requirements. For this study, we have set the threshold at 80% for the AGRAWAL dataset and 70% for the HYPER dataset. This setting is based on a 10% margin below the mean accuracy achieved by the original ARF classification using all features, accounting for the occasional variations that may occur in classification performance.

4.6 Result and Evaluation

In this section, we conduct experiments using both our proposed Feature Selection-Driven Drift Detection Method (FS-DDM) and the original Drift Detection Method (DDM) across three datasets to address concept drift. The results, as presented in Table 2, compare the classification accuracy for each dataset when using both the LR+ARF model and the original ARF classification model.

When evaluating both the AGRAWAL and HYPER datasets, we observe that classification accuracy is initially low with smaller data sizes but improves significantly as the data volume increases. As the sample size grows, the classification accuracy of the LR+ARF model approaches that of the standalone ARF classification model, which does not utilize feature selection. This trend is also illustrated in Figures 3 and 4

Table 3 below details the computational costs for each dataset when using both the LR+ARF model and the original ARF classification model.

The LR+ARF model demonstrates significant reductions in computational costs during the experiments. Although initial classification accuracy is low for datasets with small sample sizes, the LR+ARF model consumes considerably less time than the original ARF model. As the sample size increases, the gap in computational costs between the two models widens substantially.

Table 4 presents a comparison of the computational costs for two datasets using the XGB+AXGB models.

This approach demonstrates substantial advantages in processing massive datasets, exhibiting significantly lower time consumption compared to the LR+ARF model. However, the configuration of the XGB+AXGB models for the experimental scenarios was suboptimal, as evidenced by unsatisfactory DDM accuracy and frequent abrupt drops in accuracy during evaluations. We plan to refine, adjust, and further evaluate these models in future experiments.

5 Discussion

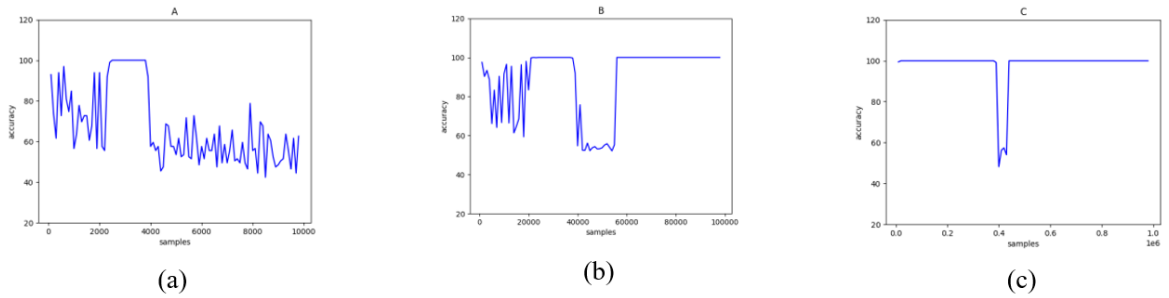
The feature selection approach demonstrated considerable promise in addressing the classification challenges inherent in drift detection problems. Particularly in environments dealing with vast amounts of data, this method significantly reduces the computational resources required. Machine learning techniques tailored for big data scenarios, such as XGBoost, have shown substantial potential for effective feature selection and data stream mining. With appropriate configuration and adjustment, the XGBoost model could also achieve impressive performance in drift detection. However, there is substantial room for improvement in the model we have presented in this work. Future efforts will focus on refining these methods to enhance their accuracy and efficiency, ensuring that they can more effectively meet the demands of dynamic and complex data environments.

6 Conclusion

In conclusion, this paper addresses the profound challenges posed by concept drift in data streams, which can critically undermine the accuracy and reliability of predictive models. The dynamic nature of big data, characterized by high dimensionality and rapid throughput, often surpasses the capabilities of traditional drift detection methods, resulting in delayed detection and consequent increases in error rates and degradation of model performance. To address these issues, we introduced an advanced drift detection model, FS-DDM, that employs multivariate analysis techniques for targeted feature selection. This strategic approach facilitates a nuanced analysis of data streams, enabling the identification of

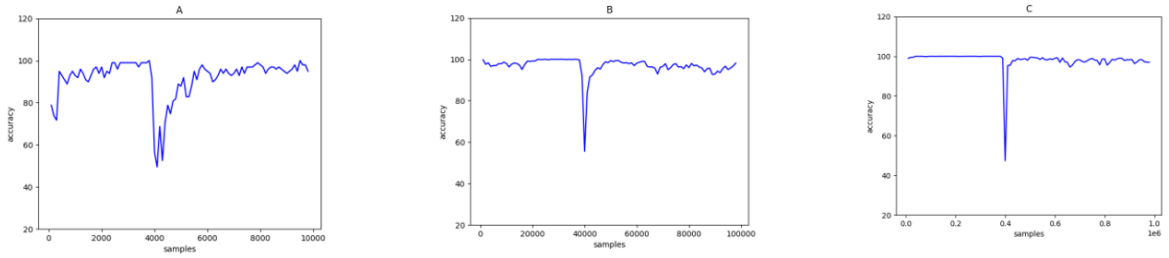
Table 2: comparison of classification accuracy of three datasets

Data type	**ARGAWAL**			**HYPER**					
Dim. Size	10			10			20		
Data Size	10,000	100,000	1,000,000	10,000	10,000	1,000,000	10,000	100,000	1,000,000
Our Accuracy	68.11%	91.06%	97.78%	69.04%	80.08%	84.95%	67.96%	74.62%	79.15%
Original DDM Accuracy	90.90%	96.52%	98.05%	82.57%	86.71%	87.77%	76.52%	80.61%	81.76%



(a) (b) (c)

Figure 3: classification accuracy record of AGRAWAL dataset with FR-DDM. (A) 10,000 sample (B) 100,000 samples (C) 1,000,000 samples



(a) (b) (c)

Figure 4: classification accuracy record of AGRAWAL dataset with original DDM. (A) 10,000 sample (B) 100,000 samples (C) 1,000,000 samples

Table 3: Comparison of computation cost of three datasets

Data type	ARGAWAL			HYPER					
Dim. Size	10			10			20		
Data Size	10,000	100,000	1,000,000	10,000	100,00	1,000,000	10,000	100,000	1,000,000
Our time cost	18.31	158.90	1905.68	13.34	238.38	1590.87	36.0	297.53	4216.53
Original DDM time Cost	24.84	221.43	2250.25	25.13	307.30	4647.52	34.0	685.24	7957.80

key features indicative of drift. By concentrating on these critical features, our model significantly enhances the efficiency and timeliness of drift detection, thus maintaining robust classification performance while substantially reducing computational demands. Experimental results affirm the efficacy of our feature-selected drift detection model, showcasing its ability to function effectively even in scenarios where data streams are voluminous and computational resources are constrained. This renders our model an invaluable tool for ensuring sustained performance in dynamic and demanding data environments, establishing a new benchmark for adaptability and precision in navigating evolving data landscapes.

Table 4: Comparison of computation cost of three dataset with XGB and AXGB model

Data type	ARGAWAL			HYPER					
Dim. Size	10			10	20				
Data Size	10,000	100,000	1,000,000	10,000	100,000	1,000,000	10,000	100,000	1,000,000
XGB+AXGB Accuracy	76.52%	86.97%	86.42%	56.52%	65.34%	64.56%	53.44%	59.02%	61.46%
XGB+AXGB time Cost	5.78	16.29	71.63	5.81	11.68	40.73	9.29	28.88	83.12

References

[1] Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. *IEEE transactions on knowledge and data engineering*, *31*(12), 2346-2363.

[2] Souza, V.M., Chowdhury, F.A., & Mueen, A. (2020). Unsupervised Drift Detection on High-speed Data Streams. *2020 IEEE International* Conference *on Big Data (Big Data)*, 102-111.

[3] Jadhav, A., & Deshpande, L.A. (2017). An Efficient Approach to Detect Concept Drifts in Data Streams. *2017 IEEE 7th International* Advance *Computing Conference (IACC)*, 28-32.

[4] Wang, P., Yu, H., Jin, N., Davies, D., & Woo, W. L. (2024). Quadcdd: A quadruple-based approach for understanding concept drift in data streams. *Expert Systems with Applications*, *238*, 122114.

[5] Gama, J., Medas, P., Castillo, G., & Rodrigues, P. (2004). Learning with drift detection. In *Advances in Artificial Intelligence–SBIA 2004: 17th Brazilian Symposium on Artificial Intelligence, Sao Luis, Maranhao, Brazil, September 29-Ocotber 1, 2004. Proceedings 17* (pp. 286-295). Springer Berlin Heidelberg.

[6] Baena-Garcıa, M., del Campo-Ávila, J., Fidalgo, R., Bifet, A., Gavalda, R., & Morales-Bueno, R. (2006, September). Early drift detection method. In *Fourth international workshop on knowledge discovery from data streams* (Vol. 6, pp. 77-86).

[7] Frias-Blanco, I., del Campo-Ávila, J., Ramos-Jimenez, G., Morales-Bueno, R., Ortiz-Diaz, A., & Caballero-Mota, Y. (2014). *Online* and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering*, *27*(3), 810-823.

[8] Pesaranghader, A., & Viktor, H. L. (2016). Fast hoeffding drift detection method for evolving data streams. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2016, Riva del Garda, Italy, September 19-23, 2016, Proceedings, Part II 16* (pp. 96-111). Springer International Publishing.

[9] *Sobolewski*, P., & Wozniak, M. (2013). Concept Drift Detection and Model Selection with Simulated Recurrence and Ensembles of Statistical Detectors. *J. Univers. Comput. Sci.*, *19*(4), 462-483.

[10] Abubakar, S.M., Sufyanu, Z., & Abubakar, M.M. (2020). A SURVEY OF FEATURE SELECTION METHODS FOR *SOFTWARE* DEFECT PREDICTION MODELS.

[11] Montiel, J., Read, J., Bifet, A., & Abdessalem, T. (2018). Scikit-multiflow: A multi-output streaming framework. *Journal of Machine Learning Research*, *19*(72), 1-5.

[12] Aggarwal, C. C., Philip, S. Y., Han, J., & Wang, J. (2003, January). A framework for clustering evolving data streams. In *Proceedings 2003 VLDB conference* (pp. 81-92).

[13] Hulten, G., Spencer, L., & Domingos, P. (2001, August). Mining time-changing data streams. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining* (pp. 97-106).